Pattern Matching using Z-Algorithm

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Rayanki Himavarshini – 2520030141
N. Sai Pranavi – 2520030514
J. Sai Spoorthi – 2520030587

Team Number:

Section:

Under the guidance of

Anuradha

Department of CSE, KLH University



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



DECLARATION

The Mini Project Report entitled "**Pattern Matching using Z-Algorithm**" is a record of Bonafide work of **Rayanki Himavarshini – 2520030141, N. Sai Pranavi – 2520030514, J. Sai Spoorthi – 2520030587** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Rayanki Himavarshini – 2520030141

N. Sai Pranavi – 2520030514

J. Sai Spoorthi – 2520030587

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled "**Pattern Matching using Z-Algorithm**" is a bonafide work done and submitted by **Rayanki Himavarshini – 2520030141, N. Sai Pranavi – 2520030514, J. Sai Spoorthi – 2520030587** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our guide, Anuradha, Department of CSE, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P. Venkateshwar Rao**, Head of the Department for Computer Science Engineering, for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted us directly or indirectly for successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Rayanki Himavarshini – 2520030141

N. Sai Pranavi – 2520030514

J. Sai Spoorthi – 2520030587

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software requirements	10
6.	Implementation	11
7.	Results & Discussion	13
8.	Conclusion & Future Scope	15
9.	References	16

1.ABSTRACT

Pattern matching is the process of locating every occurrence of a small string (the pattern) within a larger string (the text). It underlies search engines, text editors, DNA sequencing tools, and cyber-security systems. The naive character-by-character approach re-examines text after every failed attempt, giving a worst-case time complexity of O(n × m) that degrades sharply on large or repetitive data.

This project implements the Z-Algorithm, a linear-time exact string-matching technique. The pattern and text are concatenated into a single string S = P + "$" + T, and a Z-array is built in one pass, where Z[i] is the length of the longest substring starting at position i that matches a prefix of S. A pattern occurrence is found wherever Z[i] equals the length of the pattern.

By reusing previously matched information through the Z-box technique, the algorithm avoids redundant comparisons and guarantees O(n + m) time in the best, average, and worst cases. The system is implemented in Java and validated on sample text-pattern pairs, including overlapping and near-miss cases, to confirm correctness and demonstrate its efficiency over naive searching.

Team Name:

Team Members:

1. Rayanki Himavarshini, 2520030141
2. N. Sai Pranavi, 2520030514
3. J. Sai Spoorthi, 2520030587

2. INTRODUCTION

What is Pattern Matching?

Pattern matching is the process of finding all occurrences of a smaller string, called the pattern, within a larger string, called the text. It is one of the most fundamental problems in computer science and forms the backbone of search, retrieval, and analysis systems. For example, in the text "ABCABCABY", the pattern "CABCABY" is located starting at index 2.

The difficulty is not in finding a match but in finding it quickly. A naive search slides the pattern over the text one position at a time and re-compares characters after every mismatch, which costs $O(n \times m)$ time in the worst case. The Z-Algorithm answers this by concatenating the pattern and text into one string and computing a Z-array in a single pass, so that every character is examined only a constant number of times and the total running time is $O(n + m)$.

Real-World Applications

- **Search Engines –** locating keywords across billions of web pages.
- **Text Editors –** powering find, find-and-replace, and syntax highlighting.
- **DNA Sequencing –** matching genetic sub-sequences in bioinformatics.
- **Cyber Security –** intrusion detection and malware signature scanning.

The rest of this report is organised as follows: the Literature Survey reviews existing string-matching techniques; the Problem Statement and Objectives define what the project sets out to achieve; the Hardware and Software Requirements, Implementation and Results sections describe how the Z-Algorithm was built and tested in Java; and the report closes with the Conclusion, Future Scope and References.

3. LITERATURE SURVEY

Existing pattern-matching techniques were reviewed to identify a suitable linear-time approach for this project:

1. Naive (Brute-Force) Search

Slides the pattern over the text one position at a time and compares characters from left to right. It is simple to implement but re-compares characters after every mismatch, which gives an O(n × m) worst-case time on repetitive inputs and makes it unsuitable for large texts (Cormen et al., Introduction to Algorithms, 3rd Ed.).

2. Knuth–Morris–Pratt (KMP) Algorithm

Preprocesses the pattern into a failure (LPS) function so that, after a mismatch, the search resumes without re-examining characters already matched. This achieves O(n + m) time, but the failure function is harder to understand and implement correctly (Gusfield, Algorithms on Strings, Trees, and Sequences, Cambridge University Press).

3. Z-Algorithm

Builds a single Z-array over the concatenated pattern and text, fully reusing prior comparisons through the Z-box to guarantee O(n + m) time, with a simpler implementation than KMP (Landau & Vishkin, Fast String Matching with k Differences, JCSS).

Across the literature, the Z-Algorithm is consistently cited as a linear-time alternative that is easier to implement while matching KMP's efficiency — motivating its choice for this project.

Technique	Time complexity	Key idea
Naive search	O(n × m)	Re-compare after every mismatch
KMP	O(n + m)	Failure (LPS) function on the pattern
Z-Algorithm	O(n + m)	Z-array with Z-box over P + "$" + T

4. Problem Statement and Objectives

Problem Statement:

Given a text T of length n and a pattern P of length m, find all starting indices in T where P occurs, in linear time.

Problem Description:

As texts grow larger — web pages, genomes, log files — the searching method has a direct impact on speed and system responsiveness. Naive character-by-character comparison re-examines the same text repeatedly, degrades to O(n × m) on repetitive inputs, and does not scale to large real-world data.

Objectives:

1. Efficient Pattern Detection

Find every occurrence of a pattern inside a text quickly and reliably, regardless of text size.

2. Reduce Unnecessary Comparisons

Reuse previously computed match information instead of re-scanning characters already examined.

3. Improve Search Performance

Achieve linear O(n + m) time complexity so that performance scales predictably with input size.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 or above (or equivalent)
- RAM: 4 GB minimum (8 GB recommended)
- Hard Disk: 20 GB free space
- Standard keyboard, mouse, and display

Software Requirements:

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Java (JDK 8 or above)
- IDE: Eclipse, IntelliJ IDEA, or VS Code
- Any standard text editor for viewing sample input/output

6. IMPLEMENTATION

The system is implemented in Java in a single class, ZAlgorithm, made up of three modules: Z-array construction, pattern search, and result display. The overall workflow is shown in Figure 1.

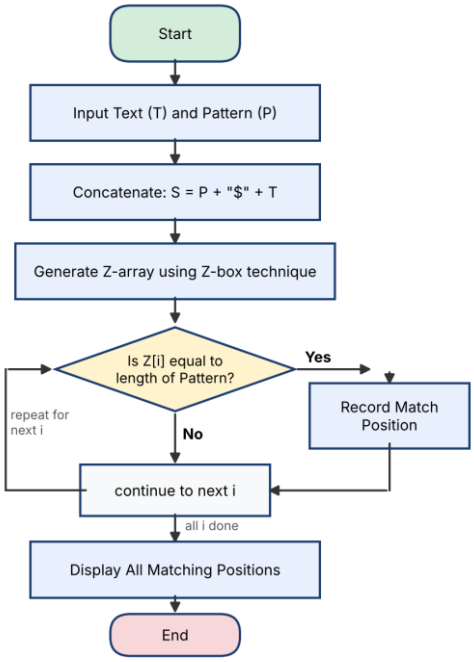


Figure 1: Flow chart of the Z-Algorithm pattern matching system

Module 1: Z-array construction (Z-box technique)

Z[i] stores the length of the longest substring starting at position i that matches a prefix of S. The window [l, r) is the Z-box: the rightmost match found so far. When i lies inside the box, Z[i] is initialised from the already computed value Z[i − l], so earlier comparisons are reused and every character of S is compared only a constant number of times.

```
static int[] buildZArray(String s) {
    int n = s.length();
    int[] z = new int[n];
    int l = 0, r = 0;                    // current Z-box [l, r)
    for (int i = 1; i < n; i++) {
        if (i < r) {
            z[i] = Math.min(r - i, z[i - l]);   // reuse earlier work
        }
        while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) {
            z[i]++;                              // extend the match
        }
        if (i + z[i] > r) {                      // update Z-box
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

Module 2: Pattern search over S = P + "$" + T

The pattern and text are joined with a separator "$" that occurs in neither string, so no match can run across the boundary. Every position i in the text part of S with Z[i] equal to the pattern length m is an occurrence, and its index in the text is i − m − 1.

```
static List<Integer> search(String text, String pattern) {
    List<Integer> positions = new ArrayList<>();
    if (pattern.isEmpty() || pattern.length() > text.length()) {
        return positions;
    }
    String s = pattern + "$" + text;             // S = P + "$" + T
    int[] z = buildZArray(s);
    int m = pattern.length();
    for (int i = m + 1; i < s.length(); i++) {
        if (z[i] == m) {
            positions.add(i - m - 1);            // index in the text
        }
    }
    return positions;
}
```

Module 3: Input and display of results

This module reads the text and the pattern from the console, calls the search module and prints every matching index together with the total number of occurrences.

```
static void display(String text, String pattern) {
    List<Integer> res = search(text, pattern);
    System.out.println("Text    : " + text);
    System.out.println("Pattern : " + pattern);
    if (res.isEmpty()) {
        System.out.println("Pattern not found in the text.");
    } else {
        StringBuilder sb = new StringBuilder("Pattern found at index ");
        for (int k = 0; k < res.size(); k++) {
            sb.append(res.get(k));
            if (k < res.size() - 1) sb.append(", ");
        }
        System.out.println(sb);
        System.out.println("Total occurrences: " + res.size());
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter text    : ");
    String text = sc.nextLine();
    System.out.print("Enter pattern : ");
    String pattern = sc.nextLine();
    display(text, pattern);
    sc.close();
}
```

7. RESULTS & DISCUSSION

Sample Run

Text: ABCDABCDABEABCDABD Pattern: ABCD

The program reports the pattern at index 0, index 4 and index 11 — a total of 3 occurrences (Figure 2).

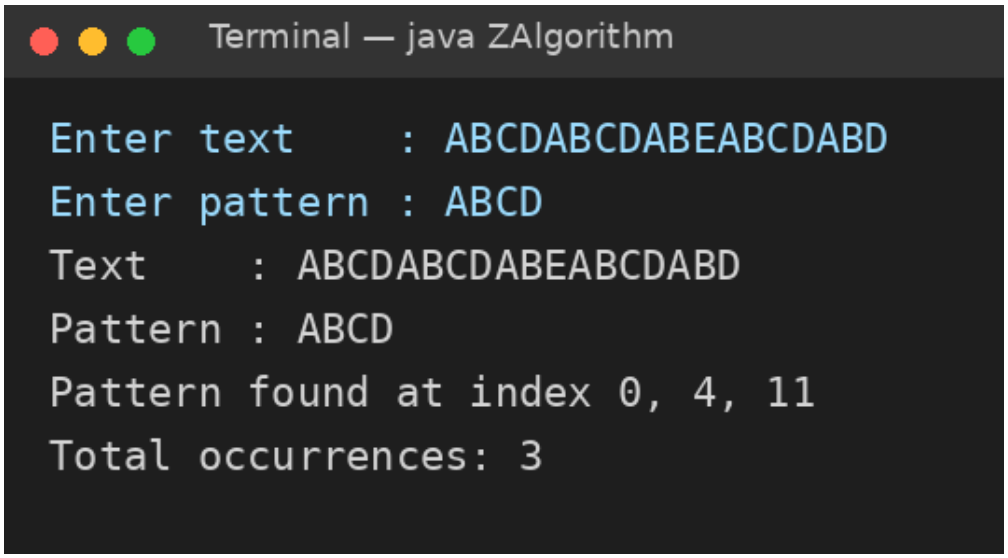


Figure 2: Output for the sample text and pattern ABCD

Observations

- Adjacent occurrences (index 0 and index 4) are both detected correctly, with no characters skipped between them.
- The near-miss region at index 8–10 ("ABE") matches only the prefix "AB" ($Z = 2 < 4$) and is correctly rejected.
- All three genuine occurrences of "ABCD" are reported, with zero missed or false matches.

Further test cases confirm the behaviour on overlapping occurrences and on a pattern that is absent from the text (Figures 3 and 4).

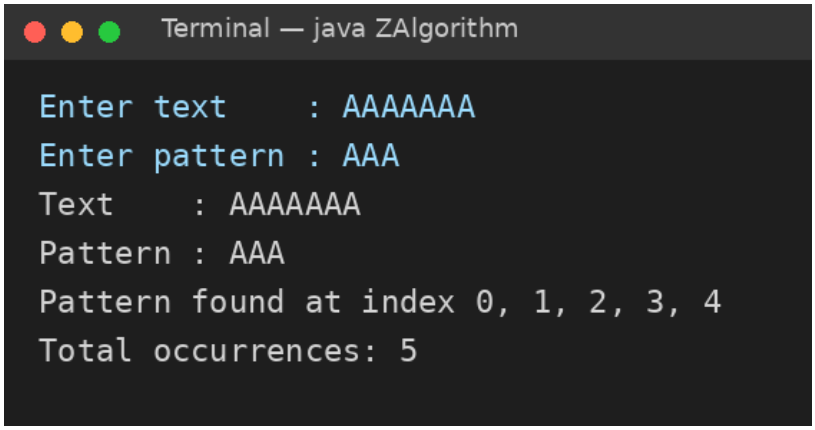


Figure 3: Overlapping occurrences – text AAAAAAA, pattern AAA

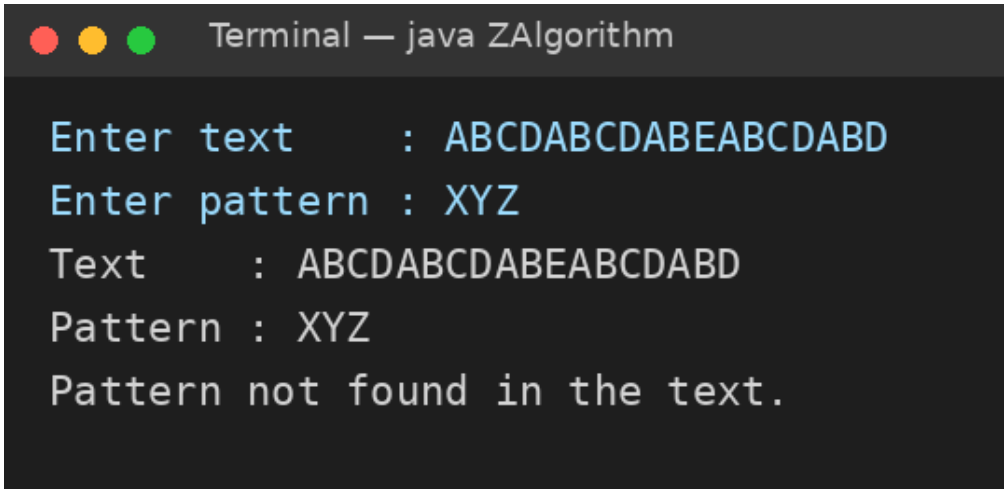


Figure 4: Pattern not present in the text

Time Complexity

The Z-Algorithm achieves O(n + m) time in the best, average, and worst cases, with O(n + m) auxiliary space — unlike naive search, there is no worst-case blow-up on repetitive inputs.

Algorithm	Best case	Average case	Worst case
Naive search	O(n)	O(n + m) typical	O(n × m)
Z-Algorithm	O(n + m)	O(n + m)	O(n + m)

8. CONCLUSION & FUTURE SCOPE

The Z-Algorithm solves exact pattern matching in guaranteed O(n + m) time by building a single Z-array over the concatenated string Pattern + "$" + Text. It outperforms naive search and matches the efficiency of KMP with a simpler design that needs only one array and a few pointers.

Implementing this project gave practical understanding of string-processing algorithms and preprocessing techniques, hands-on experience building linear-time algorithms in Java, and deeper insight into algorithmic complexity analysis and trade-offs between different string-matching approaches.

Overall, the project meets its objectives of efficient, reliable, and scalable pattern detection, and provides a solid foundation for extending the work to more advanced search applications.

Future Scope

- **GUI Application –** build a desktop or web interface so users can search visually instead of via console input.
- **Multiple Pattern Search –** extend the algorithm with generalized structures to search for many patterns at once.
- **File Search Integration –** apply the algorithm to search across entire files or directories, not just single strings.
- **Real-Time Search –** support incremental / streaming text so results update live as new data arrives.
- **AI-Based Text Analytics –** combine pattern matching with NLP techniques for smarter, context-aware text analysis.

9. REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd Ed., MIT Press, 2009.

[2] D. Gusfield, Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology, Cambridge University Press, 1997.

[3] G. M. Landau and U. Vishkin, "Fast String Matching with k Differences," Journal of Computer and System Sciences, vol. 37, no. 1, 1988.

[4] D. E. Knuth, J. H. Morris and V. R. Pratt, "Fast Pattern Matching in Strings," SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.

[5] GeeksforGeeks, "Z algorithm (Linear time pattern searching algorithm)", https://www.geeksforgeeks.org/z-algorithm-linear-time-pattern-searching-algorithm/

[6] CP-Algorithms, "Z-function and its calculation", https://cp-algorithms.com/string/z-function.html

[7] Oracle, Java SE Documentation, https://docs.oracle.com/javase/8/docs/